

# Stash — Aggregator Scenario Report

Canonical engine at web/src/lib/stashEngine.js · Web direct import · Mobile via mobile/lib/services/stash\_engine.dart.

All 74 assertions passed across 20 scenarios — web + mobile aggregators agree.

| SCENARIO                                                                                                                                                                                                         | AGGREGATED ITEMS                                                             | ASSERTIONS                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Empty user — no receipt_items at all</b><br>empty_stash<br>New user, no purchases logged. Output is the empty list.                                                                                           | (empty)                                                                      | <ul style="list-style-type: none"><li>✓ expect_count === 0</li></ul>                                                                                                                                                                                                                                                                                            |
| <b>Single item from a single receipt</b><br>single_item_single_receipt<br>One receipt with one line item. One stash entry, qty 1, totalSpent = price.                                                            | <ul style="list-style-type: none"><li>milk gallon · qty 1 · \$4.99</li></ul> | <ul style="list-style-type: none"><li>✓ expect_count === 1</li><li>✓ key="milk gallon" present</li><li>✓ milk gallon.qty === 1</li><li>✓ milk gallon.totalSpent === 4.99</li><li>✓ milk gallon.timesBought === 1</li><li>✓ milk gallon.lastDate === "2026-05-30"</li><li>✓ milk gallon.category === "grocery"</li><li>✓ milk gallon.ratingCount === 0</li></ul> |
| <b>Two receipts of the same item merge into one stash entry</b><br>two_receipts_same_item_merged<br>Same item_name bought twice on different days. One row, qty=2, totalSpent summed, lastDate = the later date. | <ul style="list-style-type: none"><li>bananas · qty 5 · \$3.95</li></ul>     | <ul style="list-style-type: none"><li>✓ expect_count === 1</li><li>✓ key="bananas" present</li><li>✓ bananas.qty === 5</li><li>✓ bananas.totalSpent === 3.95</li><li>✓ bananas.timesBought === 2</li><li>✓ bananas.lastDate === "2026-05-25"</li><li>✓ bananas.lastReceiptId === "r2"</li></ul>                                                                 |

| SCENARIO                                                                                                                                                                                                                                  | AGGREGATED ITEMS                                                                                          | ASSERTIONS                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Case-insensitive merge — "MILK" and "Milk" share a key</b><br><code>case_insensitive_merge</code><br>Different casing of the same product. Engine lowercases the key so both rows merge.                                               | <ul style="list-style-type: none"> <li>• <code>milk</code> · qty 2 · \$10.00</li> </ul>                   | <ul style="list-style-type: none"> <li>• ✓ <code>expect_count === 1</code></li> <li>• ✓ <code>key="milk" present</code></li> <li>• ✓ <code>milk.qty === 2</code></li> <li>• ✓ <code>milk.totalSpent === 10</code></li> </ul>                                                                                   |
| <b>SKU-based merge — different names same SKU collapse</b><br><code>sku_preferred_over_name_for_merge</code><br>"Coca-Cola 12oz" and "Coke 12oz" share a SKU and merge into one stash item under the SKU key.                             | <ul style="list-style-type: none"> <li>• <code>049000000443</code> · qty 2 · \$3.78 · 2 stores</li> </ul> | <ul style="list-style-type: none"> <li>• ✓ <code>expect_count === 1</code></li> <li>• ✓ <code>key="049000000443" present</code></li> <li>• ✓ <code>049000000443.qty === 2</code></li> <li>• ✓ <code>049000000443.totalSpent === 3.78</code></li> <li>• ✓ <code>049000000443.storeCount === 2</code></li> </ul> |
| <b>Returned items don't enter the stash</b><br><code>returned_items_excluded</code><br>A returned receipt_item (returned=true) is filtered before aggregation. Only non-returned rows remain.                                             | <ul style="list-style-type: none"> <li>• <code>headphones</code> · qty 1 · \$200.00</li> </ul>            | <ul style="list-style-type: none"> <li>• ✓ <code>expect_count === 1</code></li> <li>• ✓ <code>key="headphones" present</code></li> <li>• ✓ <code>headphones.qty === 1</code></li> </ul>                                                                                                                        |
| <b>Statement-import rows excluded (from_statement=true)</b><br><code>statement_imports_excluded</code><br>Receipt rows that came from a credit-card statement import aren't real product purchases. Engine filters them out of the stash. | <ul style="list-style-type: none"> <li>• <code>eggs</code> · qty 1 · \$5.00</li> </ul>                    | <ul style="list-style-type: none"> <li>• ✓ <code>expect_count === 1</code></li> <li>• ✓ <code>key="eggs" present</code></li> </ul>                                                                                                                                                                             |
| <b>Rows with blank item_name are excluded</b><br><code>empty_item_name_excluded</code><br>Corrupt rows (no name) can't render in the stash. Engine drops them silently.                                                                   | <ul style="list-style-type: none"> <li>• <code>apples</code> · qty 2 · \$2.00</li> </ul>                  | <ul style="list-style-type: none"> <li>• ✓ <code>expect_count === 1</code></li> <li>• ✓ <code>key="apples" present</code></li> <li>• ✓ <code>apples.qty === 2</code></li> </ul>                                                                                                                                |

| SCENARIO                                                                                                                                                                                                    | AGGREGATED ITEMS                                                                    | ASSERTIONS                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Latest receipt's date / store / category wins</b><br>latest_receipt_metadata_wins<br><br>Three buys across stores. lastDate = most-recent date; lastStore = that store; category = most-recent category. | <ul style="list-style-type: none"><li>• bread · qty 3 · \$9.00 · 3 stores</li></ul> | <ul style="list-style-type: none"><li>• ✓ expect_count === 1</li><li>• ✓ key="bread" present</li><li>• ✓ bread.qty === 3</li><li>• ✓ bread.lastDate === "2026-05-30"</li><li>• ✓ bread.lastReceiptId === "r3"</li><li>• ✓ bread.lastStore === "Whole Foods"</li><li>• ✓ bread.category === "bakery"</li></ul> |
| <b>Same item across stores — distinct store count is right</b><br>multi_store_distinct_count<br><br>Same SKU bought at 3 different stores. storeCount = 3, stores list contains all three.                  | <ul style="list-style-type: none"><li>• x123 · qty 3 · \$39.00 · 3 stores</li></ul> | <ul style="list-style-type: none"><li>• ✓ expect_count === 1</li><li>• ✓ key="x123" present</li><li>• ✓ x123.storeCount === 3</li></ul>                                                                                                                                                                       |
| <b>Ratings — max + avg computed across all rated rows</b><br>rating_max_and_avg<br><br>Three buys of the same item rated 3, 5, 4. ratingMax=5, ratingAvg=4.0, ratingCount=3.                                | <ul style="list-style-type: none"><li>• yogurt · qty 3 · \$15.00 · 4.0★</li></ul>   | <ul style="list-style-type: none"><li>• ✓ expect_count === 1</li><li>• ✓ key="yogurt" present</li><li>• ✓ yogurt.ratingMax === 5</li><li>• ✓ yogurt.ratingAvg === 4</li><li>• ✓ yogurt.ratingCount === 3</li></ul>                                                                                            |
| <b>Unrated rows don't affect ratingAvg</b><br>rating_null_ignored<br><br>Two buys — one rated 5, one unrated. ratingCount=1, ratingAvg=5 (the null row is excluded from the average).                       | <ul style="list-style-type: none"><li>• tea · qty 2 · \$6.00 · 5.0★</li></ul>       | <ul style="list-style-type: none"><li>• ✓ expect_count === 1</li><li>• ✓ key="tea" present</li><li>• ✓ tea.ratingCount === 1</li><li>• ✓ tea.ratingAvg === 5</li><li>• ✓ tea.ratingMax === 5</li></ul>                                                                                                        |
| <b>qty as string coerced to int</b><br>qty_strings_coerced<br><br>Supabase sometimes returns numeric columns as strings. Engine coerces qty and price defensively.                                          | <ul style="list-style-type: none"><li>• apples · qty 3 · \$3.75</li></ul>           | <ul style="list-style-type: none"><li>• ✓ expect_count === 1</li><li>• ✓ key="apples" present</li><li>• ✓ apples.qty === 3</li><li>• ✓ apples.totalSpent === 3.75</li></ul>                                                                                                                                   |

| SCENARIO                                                                                                                                                                                                              | AGGREGATED ITEMS                                                                                                                                                                      | ASSERTIONS                                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Item-level category null → falls back to receipt-level category</b><br>category_falls_back_to_receipt_category<br>When a row has no item.category but its parent receipt is categorized, that category propagates. | <ul style="list-style-type: none"> <li>• coffee filters · qty 1 · \$5.00</li> </ul>                                                                                                   | <ul style="list-style-type: none"> <li>• ✓ expect_count === 1</li> <li>• ✓ key="coffee filters" present</li> <li>• ✓ coffee filters.category === "household"</li> </ul>                                                                         |
| <b>10+ receipts of same item — qty and timesBought sum correctly</b><br>large_qty_aggregate<br>Heavy aggregation stress: 10 separate receipts each with 2 qty. qty=20, timesBought=10.                                | <ul style="list-style-type: none"> <li>• coffee beans · qty 20 · \$240.00</li> </ul>                                                                                                  | <ul style="list-style-type: none"> <li>• ✓ expect_count === 1</li> <li>• ✓ key="coffee beans" present</li> <li>• ✓ coffee beans.qty === 20</li> <li>• ✓ coffee beans.timesBought === 10</li> <li>• ✓ coffee beans.totalSpent === 240</li> </ul> |
| <b>Mixed product list — distinct items each get their own entry</b><br>mixed_items_distinct_entries<br>Receipt with 4 different products. Aggregator produces 4 separate stash items.                                 | <ul style="list-style-type: none"> <li>• apples · qty 1 · \$3.00</li> <li>• bananas · qty 1 · \$1.00</li> <li>• carrots · qty 1 · \$2.00</li> <li>• dates · qty 1 · \$4.00</li> </ul> | <ul style="list-style-type: none"> <li>• ✓ expect_count === 4</li> </ul>                                                                                                                                                                        |
| <b>Sort by 'recent' — newest lastDate first</b><br>sort_recent_orders_by_lastDate_desc<br>Three items, different lastDates. sortStash('recent') orders newest first.                                                  | <ul style="list-style-type: none"> <li>• old · qty 1 · \$1.00</li> <li>• mid · qty 1 · \$1.00</li> <li>• new · qty 1 · \$1.00</li> </ul>                                              | <ul style="list-style-type: none"> <li>• ✓ expect_count === 3</li> <li>• ✓ sort 'recent' order = [new, mid, old]</li> </ul>                                                                                                                     |
| <b>Sort by 'spent' — highest totalSpent first</b><br>sort_spent_orders_by_totalSpent_desc<br>Three items, different totals. sortStash('spent') orders most-expensive first.                                           | <ul style="list-style-type: none"> <li>• cheap · qty 1 · \$5.00</li> <li>• mid · qty 1 · \$50.00</li> <li>• pricey · qty 1 · \$500.00</li> </ul>                                      | <ul style="list-style-type: none"> <li>• ✓ expect_count === 3</li> <li>• ✓ sort 'spent' order = [pricey, mid, cheap]</li> </ul>                                                                                                                 |
| <b>filterStash by category — only items in that category</b><br>filter_by_category<br>Filter to category='produce'. Only the produce item survives.                                                                   | <ul style="list-style-type: none"> <li>• apples · qty 1 · \$3.00</li> <li>• detergent · qty 1 · \$8.00</li> </ul>                                                                     | <ul style="list-style-type: none"> <li>• ✓ expect_count === 2</li> <li>• ✓ filter category='produce' → 1</li> </ul>                                                                                                                             |

| SCENARIO                                                                                                                                                                                                    | AGGREGATED ITEMS                                                                                                                                                                             | ASSERTIONS                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>filterStash by query — case-insensitive name + sku match</b><br><code>filter_by_query</code><br>Search 'tea' matches Green Tea AND Green TEA Bags (and a SKU containing 'tea'). Detergent doesn't match. | <ul style="list-style-type: none"><li><code>green tea</code> · qty 1 · \$4.00</li><li><code>green tea bags</code> · qty 1 · \$6.00</li><li><code>detergent</code> · qty 1 · \$8.00</li></ul> | <ul style="list-style-type: none"><li>✓ <code>expect_count === 3</code></li><li>✓ <code>filter query='tea' → 2</code></li></ul> |

Cross-platform parity asserted by `mobile/test/services/stash_scenarios_test.dart`, reading the same `test-fixtures/stash-scenarios.json`.